

UNIVERSIDADE FEDERAL DE SANTA CATARINA

CENTRO DE CIÊNCIAS TECNOLÓGICAS

DEPARTAMENTO DE COMPUTAÇÃO

CURSO TECNOLOGIA DA INFORMAÇÃO E COMUNICAÇÃO

ENGENHARIA DE SOFTWARE II

DISCENTES:

Murilo Rocha,

Pedro Antonio Leão Cruvinel

# Projeto de Teste de Software

Araranguá

2026

## 1 IDENTIFICAÇÃO DO PROJETO ORIGINAL

| **Item** | **Descrição** |
|---|---|
| Nome do projeto | java-inventory-manager |
| Autor original | Nicolas Isaza Sierra (GitHub: NikoAndes) |
| URL do repositório original | https://github.com/NikoAndes/java-inventory-manager |
| Licença | MIT |
| Linguagem | Java |
| Domínio da aplicação | Controle de estoque de produtos |
| Commit utilizado como baseline | ce94ae2b8e9738410854be39ef0624b240286345 |
| Repositório do grupo | https://github.com/murilorochars/trabalho-manutencao-EngSoft-II |
| Branch de trabalho | manutencao |

## 2 JUSTIFICATIVA DA ESCOLHA

O projeto foi escolhido por ser um sistema simples, desenvolvido em Java, com uma arquitetura fácil de compreender e sem necessidade de infraestrutura complexa. Apesar da simplicidade, foram identificadas diversas possibilidades de melhoria, como a falta de uma camada Repository para organizar o acesso aos dados, além de outros problemas de manutenibilidade. Dessa forma o mesmo apresentou diversas possibilidades de melhoria como a troca da base de dados csv para o sqlite algo mais robusto e que não troca a funcionalidade do sistema preenchendo todos requisitos do trabalho requisitado.

## 3 CARACTERIZAÇÃO E COMPREENSÃO INICIAL DO SISTEMA

A exploração foi realizada seguindo os pontos solicitados no documento da professora, analisando o README, a estrutura de pastas, o código-fonte, as dependências, os testes e o histórico de commits. O projeto é um sistema de gerenciamento de estoque desenvolvido em Java, executado pelo terminal e com dados armazenados em um arquivo CSV. O código é dividido em classes responsáveis pela interface, regras de negócio, produtos e manipulação dos arquivos.

Durante a análise, identificamos que o projeto não possuía testes automatizados e apresentava problemas de manutenibilidade, como o acoplamento entre as regras de negócio e o armazenamento dos dados. Essas características permitiram identificar melhorias e planejar as refatorações e novas funcionalidades solicitadas no trabalho e o histórico de commits evidenciava apenas alterações de feat nada de melhoria ou correção dando um grande ponto de alerta que teria como aplicar técnicas de manutebildiades.

**4 BASELINE**

Antes de qualquer modificação, a versão original foi compilada, executada e registrada, conforme exigido na Etapa 3.

| **Informação** | **Resultado** |
|---|---|
| Commit da baseline | ce94ae2b8e9738410854be39ef0624b240286345 |
| Build | Sucesso |
| Quantidade de testes | 0 |
| Testes executados com sucesso | Não aplicável (inexistentes) |
| Execução da aplicação | Sucesso |
| Ambiente | OpenJDK 25.0.4 (Temurin), Windows 11 |

Os comandos utilizados foram:

```
javac -d bin -sourcepath src src/Main.java
java -cp bin Main
```

A compilação concluiu sem erros ou avisos, e a aplicação apresentou o menu e

listou os dez produtos do arquivo data/inventory.csv. A saída da listagem e a do relatório de estoque baixo foram registradas e utilizadas como referência de comparação em todas as verificações posteriores.

## 5 PROBLEMAS DE MANUTENIBILIDADE IDENTIFICADOS

A análise combinou leitura do código e execução da aplicação com entradas propositalmente inadequadas. Foram identificados cinco problemas.

### 5.1 Acoplamento do serviço ao formato de arquivo

**Localização:** InventoryService, construtor e método save().

**Evidência:** a classe declara import util.CsvHandler e invoca CsvHandler.load(dataFile) e CsvHandler.save(dataFile, products) por meio de chamadas estáticas diretas. O construtor recebe o caminho de um arquivo.

**Consequência:** a classe que concentra a regra de negócio conhece o meio de armazenamento. Como o próprio README planeja substituir o CSV por SQLite, essa estrutura obrigaria a reescrever a regra de negócio para realizar uma mudança de infraestrutura. Além disso, impede testar o serviço sem gravar arquivos em disco. Caracteriza violação do Princípio da Inversão de Dependência e constitui dívida técnica arquitetural, pois o custo da evolução já prevista cresce em função dessa decisão.

**Possível tratamento:** *Extract Interface* com injeção pelo construtor.

### 5.2 Ausência de testes automatizados

**Localização:** o projeto como um todo.

**Evidência:** não existe diretório de testes nem pom.xml, embora o README afirme o contrário.

**Consequência:** não há mecanismo para demonstrar que uma modificação preservou o comportamento existente. Qualquer intervenção depende de verificação

manual, o que eleva o risco de regressões não detectadas.

**Possível tratamento:** escrever testes de caracterização antes de refatorar.

### 5.3 Ausência de validação das regras de negócio

**Localização:** construtor e *setters* de Product; InventoryService.addProduct; Main.addProduct.

**Evidência:** os valores são atribuídos diretamente, sem verificação. A execução confirmou que o sistema aceita nome vazio, preço negativo e quantidade negativa; aceita dois produtos idênticos; encerra com NumberFormatException quando o preço é digitado como texto; e, se o nome contiver uma vírgula, o arquivo CSV é corrompido e a aplicação não consegue mais ser iniciada.

**Consequência:** estados inválidos são gravados na base e, no caso da vírgula, a falha é permanente, pois o dado corrompido impede a inicialização.

**Possível tratamento:** *guard clauses* no construtor. Observa-se que essa alteração modifica o comportamento observável e, portanto, não se enquadra como refatoração.

### 5.4 Números e cadeias mágicos

**Localização:** InventoryService e CsvHandler.

**Evidência:** o valor 5, utilizado como limite de estoque baixo, aparece sem identificação; o prefixo "P" dos identificadores aparece em computeNextId() e, novamente, como "P%03d" em addProduct.

**Consequência:** a mesma regra reside em dois pontos distintos. Alterar o formato do identificador exige localizar todas as ocorrências, e o esquecimento de qualquer uma delas produz identificadores inconsistentes. É o "juro" característico da dívida técnica: um custo recorrente cobrado a cada modificação futura.

**Possível tratamento:** *Replace Magic Number with Symbolic Constant*.

### 5.5 Entidade de domínio conhece o formato de persistência

**Localização:** Product.toCsv() e Product.fromCsv().

**Evidência:** a entidade de domínio constrói e interpreta linhas de CSV. O conhecimento do formato está repartido entre duas classes: o cabeçalho pertence ao CsvHandler e a ordem das colunas, ao Product.

**Consequência:** substituir o meio de persistência obriga a modificar o modelo de domínio, que nada tem a ver com arquivos. Caracteriza *Feature Envy* e mistura de responsabilidades.

**Possível tratamento:** *Move Method*.

### 5.6 Código morto

Adicionalmente, registrou-se que os métodos findById, filterByCategory e setLowStockThreshold do InventoryService nunca são invocados. O último deles é particularmente significativo: a capacidade de configurar o limite de estoque existe no código e é anunciada no README ("Low-stock alert system with configurable threshold"), mas nenhuma parte da interface a expõe ao usuário.

## 6 REFATORAÇÕES REALIZADAS

Foram selecionados três problemas e aplicadas as refatorações correspondentes, sem alteração intencional do comportamento observável.

### 6.1 R1 Extração da interface de repositório (commit d0684a7)

Trata o problema descrito em 5.1.

**Antes:**

```
import util.CsvHandler;
```

```
public class InventoryService {
    public InventoryService(String dataFile) {
        this.dataFile = dataFile;
        this.products = CsvHandler.load(dataFile);
    }

    private void save() {
        CsvHandler.save(dataFile, products);
    }
}
```

**Técnica:** *Extract Interface* com injeção de dependência pelo construtor.

**Depois:**

```
public interface ProductRepository {
    List<Product> findAll();
    void saveAll(List<Product> products);
}

public class InventoryService {
    private final ProductRepository repository;

    public InventoryService(ProductRepository repository) {
        this.repository = repository;
        this.products = repository.findAll();
    }

    private void save() {
        repository.saveAll(products);
    }
}
```

A classe CsvProductRepository passou a implementar a interface, delegando ao CsvHandler já existente, e o Main passou a construir o serviço com new InventoryService(new CsvProductRepository(DATA_FILE)).

**Justificativa:** a regra de negócio deixa de depender do meio de armazenamento e passa a depender de uma abstração. Essa refatoração tem dois efeitos práticos, ambos confirmados posteriormente neste trabalho: tornou possível trocar o CSV pelo SQLite alterando uma linha do Main, e tornou possível testar o InventoryService com um repositório em memória, sem acesso a disco.

### 6.2 R2 Substituição de números mágicos por constantes (commit 04a0551)

Trata o problema descrito em 5.4.

**Antes:**

```
this.lowStockThreshold = 5;
...
Integer.parseInt(p.getId().replace("P", ""))
...
String id = String.format("P%03d", nextId++);
```

**Técnica:** *Replace Magic Number with Symbolic Constant*.

**Depois:**

```
private static final int LIMITE_ESTOQUE_BAIXO = 5;
private static final String PREFIXO_ID = "P";
...
this.lowStockThreshold = LIMITE_ESTOQUE_BAIXO;
...
Integer.parseInt(p.getId().replace(PREFIXO_ID, ""))
...
String id = String.format(PREFIXO_ID + "%03d", nextId++);
```

**Justificativa:** o identificador nomeado explicita o significado do valor, e o prefixo dos identificadores passa a ter um único ponto de definição. Elimina-se o risco de alteração parcial descrito em 5.4.

### 6.3 R3 Movimentação da conversão de formato (commit 9e80d12)

Trata o problema descrito em 5.5.

**Antes:** Product declarava os métodos toCsv() e fromCsv(String).

**Técnica:** *Move Method*.

**Depois:** ambos foram transferidos para CsvHandler, com visibilidade privada, junto das demais operações de arquivo:

```
private static String toCsv(Product p) {
    return p.getId() + "," + p.getName() + "," + p.getCategory()
        + "," + p.getPrice() + "," + p.getQuantity();
```

```
}

private static Product fromCsv(String line) {
    String[] parts = line.split(",");
    return new Product(parts[0], parts[1], parts[2],
            Double.parseDouble(parts[3]), Integer.parseInt(parts[4]));
}
```

**Justificativa:** o conhecimento sobre o formato do arquivo passa a residir em uma única classe. A entidade de domínio volta a tratar apenas do produto.

### 6.4 Verificação da preservação de comportamento

No momento em que as refatorações foram realizadas, o projeto ainda não possuía testes automatizados. A preservação de comportamento foi verificada por recompilação e execução da aplicação, comparando-se a saída de cada opção do menu com a saída registrada na baseline. Confirmaram que o comportamento das operações originais permanece inalterado após as três refatorações.

## 7 SOLICITAÇÃO DE MANUTENÇÃO PERFECTIVA

Foram especificadas duas solicitações antes de qualquer implementação.

### 7.1 SM-01 Migração do armazenamento de CSV para SQLite

**Necessidade.** O inventário é armazenado em um arquivo CSV, formato sem estrutura ou tipagem, suscetível à corrupção por um simples caractere de vírgula no nome de um produto. A substituição por um banco de dados relacional embarcado melhora a organização e o gerenciamento dos dados, e corresponde à melhoria já prevista pelo autor do projeto.

**Comportamento esperado.** Todas as operações existentes devem produzir a mesma saída observada na baseline, passando a ler e gravar em uma tabela products do banco data/inventory.db. Os dez produtos existentes devem ser transferidos. Do ponto de vista do usuário, não deve haver diferença perceptível de comportamento.

**Regras e restrições.**

- O InventoryService não deve ser alterado, pois já depende da abstração ProductRepository introduzida em R1;
- o CsvProductRepository e o arquivo CSV devem ser preservados, servindo como origem dos dados na carga inicial;
- o esquema da tabela deve espelhar as colunas do CSV, com id como chave primária;
- é necessário incorporar o driver JDBC do SQLite.

**Critérios de verificação.** (i) a aplicação compila e executa sem depender do arquivo CSV; (ii) a listagem apresenta os mesmos dez produtos da baseline, na mesma ordem; (iii) um produto cadastrado permanece no inventário após o encerramento e a reabertura da aplicação; (iv) um produto removido não reaparece; (v) o arquivo gerado é um banco SQLite válido; (vi) nenhuma linha do InventoryService foi modificada.

### 7.2 SM-02 Parametrização do limite de estoque baixo

**Necessidade.** O alerta de estoque baixo utiliza o valor fixo 5 para todos os produtos. Esse valor não é adequado a inventários heterogêneos: um componente adquirido em lotes de centenas de unidades não está em situação crítica com seis unidades, ao passo que um item de alto custo pode exigir alerta a partir de dez. Na baseline, alterar esse limite exige recompilar o sistema. Observa-se ainda que o README anuncia a funcionalidade como existente e que o método setLowStockThreshold já está implementado, porém inacessível (ver 5.6).

**Comportamento esperado.** Uma nova opção de menu permite ao usuário informar o limite. Após a alteração, o relatório de estoque baixo e a contagem exibida nas estatísticas passam a considerar o novo valor, e a tela de alertas informa o limite em uso.

**Regras e restrições.**

- O limite padrão permanece 5, de modo a não alterar o comportamento de quem não utilizar a nova opção;
- o limite não pode ser negativo e deve ser um número inteiro;
- entradas inválidas são recusadas, o limite anterior é preservado e a aplicação não é encerrada;
- o limite vale para a sessão em execução; sua persistência foi deliberadamente excluída do escopo;
- a regra de comparação já existente em Product.isLowStock(int) deve ser reaproveitada, e não duplicada.

**Critérios de verificação.** (i) sem uso da nova opção, os alertas listam os mesmos três produtos da baseline; (ii) com limite 2, apenas o produto de estoque 2 é listado; (iii) com limite 0, nenhum produto é listado; (iv) com limite 15, a lista é ampliada; (v) limite negativo é recusado com mensagem, preservando-se o valor anterior; (vi) entrada não numérica é recusada sem encerrar a aplicação.

## 8 ANÁLISE DE IMPACTO

A análise foi registrada antes da implementação e orientou-se pela questão: se as mudanças forem realizadas, quais partes do software provavelmente precisarão ser modificadas ou verificadas

**Quadro 1 Impacto previsto**

| **Elemento** | **SM-01 (SQLite)** | **SM-02 (limite)** |
|---|---|---|
| Main.java | Modificar: *import*, constante do arquivo e instanciação do repositório | Modificar: nova opção de menu e método correspondente |
| service/InventoryService.java | Não deve ser alterado | Modificar: validação e método de leitura do limite |
| repository/ProductRepository.java | Não muda; a interface já atende | Não muda |
| repository/SqliteProductRepository.java | Criar | Não se aplica |

| Elemento | SM-01 (SQLite) | SM-02 (limite) |
|---|---|---|
| repository/CsvProductRepository.java | Preservar; passa a ser usado apenas na carga inicial | Não muda |
| util/CsvHandler.java | Preservar | Não muda |
| model/Product.java | Não muda | Não deve mudar; isLowStock(int) já é parametrizado |
| Dados | Migrar de inventory.csv para inventory.db; requer utilitário de carga | Não afetado |
| Dependências | Nova: driver JDBC sqlite-jdbc | Nenhuma |
| Build | run.sh deve incluir o driver no *classpath* | Não afetado |
| Testes | Caixa-preta | Caixa-preta |

A introdução de uma dependência externa. É o único ponto em que o projeto deixa de ser compilável exclusivamente com a JDK, o que afeta as instruções de execução e a reprodutibilidade do ambiente por terceiros. A refatoração R1 deve fazer com que a substituição do meio de armazenamento custe a alteração de uma única linha significativa no Main, sem qualquer modificação na regra de negócio. Caso o InventoryService precise ser alterado, a refatoração não terá cumprido seu objetivo.

## 9 DESCRIÇÃO DA IMPLEMENTAÇÃO

A implementação foi registrada em três commits na branch manutencao.

### 9.1 Commit d384432  migração para SQLite

Foi criada a classe SqliteProductRepository, implementando a mesma interface já utilizada pela versão em CSV:

```
public class SqliteProductRepository implements ProductRepository {
    private final String url;

    public SqliteProductRepository(String dbFile) {
        this.url = "jdbc:sqlite:" + dbFile;
```

```
        criarTabela();
    }

    private void criarTabela() {
        String sql = "CREATE TABLE IF NOT EXISTS products ("
                + "id TEXT PRIMARY KEY,"
                + "name TEXT NOT NULL,"
                + "category TEXT,"
                + "price REAL,"
                + "quantity INTEGER)";
        ...
    }
}
```

O método saveAll remove e reinsere todos os registros dentro de uma transação. Essa decisão preserva a semântica que o CSV já possuía — o arquivo era integralmente reescrito a cada gravação — e evita alterar o contrato da interface, o que propagaria mudanças para o InventoryService.

A alteração no Main restringiu-se a três linhas:

```
// antes
import repository.CsvProductRepository;
private static final String DATA_FILE = "data/inventory.csv";
service = new InventoryService(new CsvProductRepository(DATA_FILE));

// depois
import repository.SqliteProductRepository;
private static final String DATA_FILE = "data/inventory.db";
service = new InventoryService(new SqliteProductRepository(DATA_FILE));
```

A transferência dos dados foi realizada pelo utilitário util.MigracaoCsvSqlite, executado uma única vez, que lê por meio do repositório CSV e grava por meio do repositório SQLite, reaproveitando integralmente o código existente:

```
$ java -cp "bin;lib/sqlite-jdbc-3.46.1.3.jar" util.MigracaoCsvSqlite
Migrados 10 produtos de data/inventory.csv para data/inventory.db
```

### 9.2 Commit 2636e67 limite de estoque configurável

No InventoryService, o método setLowStockThreshold, até então código morto, foi aproveitado, recebeu uma *guard clause* e passou a contar com o método de leitura

correspondente:

```
public void setLowStockThreshold(int threshold) {
    if (threshold < 0) {
        throw new IllegalArgumentException("O limite nao pode ser negativo.");
    }
    this.lowStockThreshold = threshold;
}

public int getLowStockThreshold() {
    return lowStockThreshold;
}
```

No Main, foi acrescentada a opção 7 do menu e o método que a atende, com tratamento distinto para entrada não numérica e para valor fora da faixa admitida:

```
private static void setLowStockThreshold() {
    System.out.printf("Current threshold: %d%n", service.getLowStockThreshold());
    System.out.print("New threshold: ");
    try {
        service.setLowStockThreshold(Integer.parseInt(scanner.nextLine().trim()));
        System.out.printf("Threshold updated to %d.%n", service.getLowStockThreshold());
    } catch (NumberFormatException e) {
        System.out.println("Invalid value. Enter a whole number.");
    } catch (IllegalArgumentException e) {
        System.out.println(e.getMessage());
    }
}
```

O método showLowStock() passou a exibir o limite em uso. A classe Product não foi modificada: o método isLowStock(int threshold) já recebia o limite por parâmetro, o que permitiu reaproveitar a regra sem duplicá-la.

## 10 TESTES E EVIDÊNCIAS DE REGRESSÃO

Os testes foram feito com base em técnica de caixa preta observamos entrada e saida para ver se comportava igual o baseline original do projeto.eto. O objetivo principal foi verificar a compilação correta, a execução bem-sucedida do sistema integrado ao SQLite, o tratamento de erros no console e a preservação das funcionalidades após as refatorações e manutenções (SM-01 e SM-02)

### 10.1 Construção da suíte

Para validar o comportamento do sistema após a migração para o banco de dados SQLite e a implementação do limite de estoque baixo configurável, foram executados os seguintes cenários de teste diretamente no terminal:

**Teste 1: Listagem Inicial de Produtos (Opção 1 do Menu)**

- **Entrada:** Seleção da opção 1 ("List all products") logo após a inicialização.
- **Saída Observada:** O sistema carregou e exibiu com sucesso os 10 produtos iniciais migrados do arquivo CSV para o banco SQLite, mantendo os códigos de identificação (P001 a P010) e os dados de categoria, preço e estoque idênticos aos da *baseline*, além de um produto de teste adicional (P011).

**Teste 2: Consulta e Atualização do Limite de Estoque Baixo (Opção 7 do Menu)**

- **Entrada:** Seleção da opção 7 ("Set low-stock threshold"), informando um novo limite numérico válido (22).
- **Saída Observada:** O console exibiu o limite anterior, aceitou a nova entrada e retornou a mensagem de confirmação: Threshold updated to 22..

**Teste 3: Validação do Relatório de Alertas com Novo Limite (Opção 5 do Menu)**

- **Entrada:** Seleção da opção 5 ("Low-stock alerts") após definir o limite como 22.
- **Saída Observada:** O sistema recalculou dinamicamente os itens abaixo do limite e listou corretamente todos os produtos cujo estoque era inferior a 22 unidades (incluindo itens como o P001 com estoque 15 e P003 com estoque 4).

**Teste 4: Teste de Resiliência a Entradas Inválidas e Limites Negativos**

- **Entrada:** Inserção de valores não numéricos ou números negativos ao tentar configurar o limite de estoque baixo na Opção 7.
- **Saída Observada:** O sistema barrou a entrada incorreta com mensagens amigáveis no console (Invalid value. Enter a whole number. ou a exceção tratada de limite negativo), preservando o valor anterior do limite e mantendo a aplicação em execução sem interrupções abruptas (*crashes*).

## 12 DIFICULDADES ENCONTRADAS

A primeira dificuldade encontrada foi identificar que o README não correspondia à estrutura real do repositório. A documentação mencionava o uso de Maven, JUnit 5 e uma classe de teste, além de apresentar uma estrutura de diretórios diferente da existente. Como consequência, o grupo perdeu tempo procurando arquivos inexistentes. Essa experiência demonstrou a importância de verificar as informações da documentação com base no código-fonte, especialmente durante a manutenção de software de terceiros.

### 12.1 Ausência de testes automatizados

As refatorações realizadas na Etapa 5 ocorreram sem a presença de testes automatizados, tornando necessária a execução manual da aplicação para verificar os resultados. Essa limitação aumentou o tempo de validação e dificultou a identificação de possíveis regressões. A elaboração de testes de caracterização antes das refatorações, conforme discutido na seção 5.2, teria proporcionado maior segurança durante as modificações.

### 12.2 Desafios da mudança de infraestrutura

A migração do armazenamento em CSV para SQLite demonstrou que mudanças de infraestrutura envolvem aspectos que vão além do código-fonte. Foi necessário configurar a biblioteca JDBC, ajustar o *classpath* e adaptar os procedimentos de compilação e execução, além de considerar questões relacionadas ao controle de versão e à versão da JDK. Esses ajustes representaram uma parcela significativa do trabalho, evidenciando a importância de considerar as dependências e configurações do ambiente durante a manutenção de software.

### 12.3 Definição dos limites da intervenção

Alguns problemas identificados na Etapa 4 permaneceram sem tratamento,

especialmente a ausência de validação descrita na seção 5.3, que pode comprometer a integridade dos dados armazenados. Embora sua correção fosse relevante, o grupo optou por não ampliar o escopo da atividade, priorizando os objetivos inicialmente estabelecidos. Essa decisão reforçou a importância de distinguir as refatorações, que buscam melhorar a estrutura interna do software sem alterar seu comportamento observável, das modificações funcionais que devem ser avaliadas separadamente.

### 12.4 Interpretação da intenção do autor

Alguns trechos do código exigiram uma análise mais cuidadosa, como o método `setLowStockThreshold`, que estava implementado, mas não era utilizado. Em vez de removê-lo como código aparentemente desnecessário, o grupo decidiu aproveitá-lo na solicitação SM-02, considerando que o README já mencionava a funcionalidade de alerta de estoque baixo. Essa decisão baseou-se na interpretação das evidências disponíveis para compreender a intenção original do desenvolvedor, situação comum na manutenção de sistemas de terceiros.

## INSTRUÇÕES DE EXECUÇÃO

Requisitos: JDK 17 ou superior. O driver sqlite-jdbc acompanha o repositório, em lib/.

Executar a aplicação:

```
./run.sh
```

Executar a suíte de testes:

```
./test.sh
```

Alternativamente, sem os scripts, no Windows:

```
javac -cp "lib/sqlite-jdbc-3.46.1.3.jar" -d out -sourcepath src src/Main.java
```

java --enable-native-access=ALL-UNNAMED -cp "out;lib/sqlite-jdbc-3.46.1.3.jar" Main

## HISTÓRICO DE COMMITS DA MANUTENÇÃO

| Commit | Etapa | Mensagem |
|---|---|---|
| d0684a7 | 5 (R1) | refatoração: cria repositório para separar o service do CSV |
| 04a0551 | 5 (R2) | refatoração: troca números mágicos por constantes no InventoryService |
| 9e80d12 | 5 (R3) | refatoração: move toCsv e fromCsv do Product para o CsvHandler |
| d384432 | 8 (SM-01) | refactor: migra a persistencia de CSV para banco SQLite |
| 2636e67 | 8 (SM-02) | feat: permite ao usuario configurar o limite de estoque baixo |